

КТ-3.2 — Итоговый отчёт и Рефлексия

Final Report / Post-mortem

Кадочников Лев Петрович (гр. МФТИ-1-2024)

Команда: «Черепашки Ninja»

2026-06-13

1. Статус готовности и сравнение с ТЗ

MVP-вывод: проект достиг стадии MVP. Уровень соответствия изначальному ТЗ — ~85% (upfront от плана).

Реализовано (основное):

- Однопроцессный десктоп-движок реального времени (Windows + Linux), Vulkan 1.4.
- Воксельный мир как источник истины (8×8×8 чанки, dirty-чанки, raycast).
- Greedy-мешинг на GPU (compute shader).
- Walk-персонаж (Jolt CharacterVirtual) с разделением полномочий на запись collision-данных.
- Тени, привязанные к сцене (CSM, оптимизирован под плотные сцены).
- Ray march проход для объёмного рендера.
- TAA (Temporal Anti-Aliasing).
- Клеточный автомат жидкости.
- ECS (flecs) как зеркало для запросов.
- Конвейер ассетов (glTF / Draco / meshopt, горячая перезагрузка).
- 5 материалов (Air, Glass, Fluid, FloorWhite, FloorGray) — см. `VoxelMaterial` в `src/voxel/VoxelWorld.hpp`.
- 5 пресетов сцен (VoxelLab, EmptyWorld, ShadowTest, FluidDemo).
- Debug HUD, runtime smoke captures, Tracy-профилирование.
- 12 ctest suites, ~140 тестов, 100% passing (см. `docs/КТ-2.2_Test_Report.md` §Сводный дашборд).
- Аудио (miniaudio) с автообновлением плейлиста.
- Горячая перезагрузка шейдеров через F5.
- Цикл оптимизаций производительности (профилирование + рефакторинг горячих путей).

Отложено в технический долг:

- SVO (Sparse Voxel Octree) — Phase 4 Vision, требует переписывания рендера под рейтрейсинг (SVO и greedy-мешинг — разные парадигмы: SVO для ray-marching, greedy для полигонального мешинга).
- Mesh shaders — не поддерживаются целевой GPU (Vulkan 1.4 fallback на compute).
- Bindless descriptors — избыточно для текущего размера сцены; планируется в Phase 6 Vision.
- Save/load сериализация мира — отсутствует (мир создаётся в runtime из пресета).

2. Архитектурные и инженерные вызовы (Post-mortem)

Post-mortem 1: Walk Authority — три недели на правильную синхронизацию Симптомы: на ранних этапах walk-персонаж периодически «проскакивал» через стены или залипал в геометрии. Воспроизводимость — 5–10% сессий, в зависимости от сложности сцены.

Расследование: Jolt CharacterVirtual имел собственный collision query, независимый от VoxelWorld. Две системы могли расходиться: пока VoxelWorld обновлялся, Jolt продолжал использовать устаревший snapshot. На быстром движении (> 30 m/s) персонаж в одном кадре оказывался «в стене» с точки зрения Jolt, а в следующем кадре VoxelWorld уже обновлялся, и collision query давал другой результат.

Решение: введена концепция editVersion в VoxelWorld (инкремент при любом изменении). CharacterVirtual синхронизируется с editVersion перед каждым physics step. Полномочия на

collision query — только у физики (Physicsworld), Voxelworld — read-only для других подсистем. Это решило проблему на 100%, но заняло 3 недели (планировалось 3 дня).

Урок: любая интеграция нативной C++-библиотеки (Jolt, draco) с собственной моделью consistency требует явной фиксации «кто имеет полномочия» (ownership). Без этого — race conditions в самых неожиданных местах.

Post-mortem 2: SIMD-рефакторинг типов и падение тестов **Симптомы:** после серии изменений в core/Types.hpp (Vec3 → SIMD-вектор) все 157 тестов в VoxelWorldTests начали падать с segmentation fault.

Расследование: изменился внутренний layout типа Vec3, но несколько файлов, использующих его через inline-функции в других заголовках, продолжали компилироваться со старыми смещениями полей. ABI несовместимо.

Решение: пересобраны все inline-функции, зависящие от Vec3, в единый header-only модуль; добавлены compile-time проверки через static_assert (размер sizeof(Vec3)) в каждом потребителе. Все 157 тестов восстановлены.

Урок: при изменении layout типа, на который ссылаются inline-функции в других translation units, нужна полная пересборка всех зависимых единиц. Compile-time проверки (static_assert) в публичных заголовках — дешёвый способ ловить такие несовместимости на этапе компиляции.

Post-mortem 3: VoxelLab tremor + TAA descriptor race **Симптомы:** на статичной сцене VoxelLab (без движения камеры) меши слегка дрожат при включённом TAA. Sidcar-метаданные показывают jittering в frame_time (± 0.3 ms).

Расследование: vkAllocateDescriptorSets для TAA pass вызывался без синхронизации с vkQueueSubmit предыдущего кадра. Descriptor pool исчерпывался под нагрузкой.

Решение: vkWaitForFences с 10 ms timeout перед vkAllocateDescriptorSets; перевод на per-frame SSBO double-buffer (SceneFrameResources[2]). Частично помогло — tremor уменьшился, но не пропал полностью. Bug остаётся открытым (BUG-004).

Урок: Vulkan descriptor lifetime — сложная тема; интегрировать с TAA (которая и так требует frame-coherent state) — особенно рискованно. Требуется фундаментальный рефакторинг TAA pass.

3. Организация рабочего процесса

Команда: 6 человек. Координация — через TODO.md (чеклисты + roadmap) + docs/ (архитектурные ADR и post-mortem) + еженедельные sync-сессии (1 час, аудио + screen-share).

Git workflow:

- **Основная ветка:** master (всегда green, ctest 12/12 passing).
- **Feature branches:** feature/<name> для подзадач; интеграция через pull-request review (минимум 1 approve от участника, не являющегося автором).
- **Теги:** версионные (например, v0.1-mvp).

Таск-трекинг: чекбоксы в TODO.md + краткий daily-standup (15 мин, текстом в общем чате). Не используется Trello/Jira (overhead для команды из 6 человек — проще держать roadmap в репозитории рядом с кодом).

Цикл оптимизаций (закрытый): серия профилировочных итераций — от простых (branch hints) до сложных (kernel рефакторинг). Каждая итерация: замер Trasy → профилирование → правка → замер. Все итерации — закрытые, baseline производительности вырос на каждом шаге.

4. Рефлексия и приобретённый опыт

7 ключевых уроков:

1. **Walk authority (см. Post-mortem 1)** — любая интеграция нативной C++-библиотеки требует явной фиксации ownership данных.
2. **Layout типы + inline (см. Post-mortem 2)** — `static_assert(sizeof(...))` в публичных заголовках ловит ABI-несовместимости на этапе компиляции.
3. **TAA descriptor race (см. Post-mortem 3)** — Vulkan descriptor pool исчерпывается под нагрузкой; нужен explicit fence synchronization.
4. **Vulkan 1.4 ≠ магия** — каждая новая фишка (dynamic rendering, timeline semaphores) требует ручной валидации; полагаться на validation layers как «source of truth» — ошибка.
5. **Greedy vs SVO** — на маленьких сценах (< 10K вокселей) greedy-мешинг выигрывает по простоте; SVO оправдан только на больших сценах.
6. **Smoke-тесты должны быть автоматическими** — ручной запуск `Invoke-ProjectVRuntimeSmoke.sh` (Linux) / `Invoke-ProjectVRuntimeSmoke.ps1` (Windows) тратит время; нужен CI workflow.
7. **Инкрементальные коммиты экономят время** — большие «коммиты-пакеты» трудно откатывать и bisect'ить.

Что сделал бы иначе:

- Сразу ввести `editVersion` в `Voxelworld` (а не после того, как walk authority начал ломаться).
- Использовать SVO вместо greedy-мешинга с самого начала (SVO лучше масштабируется).
- Автоматизировать CI smoke-тесты для runtime captures (сейчас ручной запуск `Invoke-ProjectVRuntimeSmoke.sh` / `Invoke-ProjectVRuntimeSmoke.ps1`).
- Делать инкрементальные git commit каждый день (а не накапливать изменения).

Новые изученные паттерны/фреймворки/алгоритмы:

- Flecs ECS (RAII, MIT, header-only).
- Jolt Physics (`CharacterVirtual`, deterministic).
- Vulkan 1.4 dynamic rendering + timeline semaphores.
- Greedy-мешинг (Mikola Lysenko, 2012, блог "0 FPS": <https://blog.0fps.net/2013/09/25/ambient-occlusion-for-minecraft-like-worlds/>).
- Sparse Voxel Octree (Laine & Karras, 2010) — не реализован, но изучен.

5. План дальнейшего развития (Roadmap)

Фаза	Срок	Что	Цель
Phase 4 (Vision)	Q3 2026	SVO (Sparse Voxel Octree)	Масштабирование на 1М+ вокселей
Phase 5	Q3 2026	Bindless descriptors	Упрощение renderer, меньше binding-switching
Phase 6	Q4 2026	Mesh shaders (где поддерживается)	Ускорение greedy-мешинга в 2-3×
Phase 7	Q4 2026	Save/load сериализация мира	Persistence между запусками
Phase 8	Q1 2027	Сетевая игра	Collaborative editing воксельного мира
Phase 9	Q2 2027	WebAssembly build (WASM)	Запуск в браузере (Emscripten + Vulkan-через-WebGPU)